DNS Resolver using AF_XDP

A recursive resolver with a kernel-bypass client path, compared against the Linux socket path

Divisht	Ankit Saha	Jahan Zaib	Mateen Sharief	Adil Khan N	Aditya Kumar
24110112	*24110042*	*24110140*	*24110197*	*23110013*	*24110140*

I. Introduction

Almost every connection on the Internet begins with a DNS lookup. A recursive resolver answers these lookups on behalf of many clients. Each query and each answer is a small UDP packet, so a busy resolver spends much of its time moving packets rather than doing DNS work. On Linux, every packet that reaches an application through a normal socket passes through the driver, the IP and UDP layers and the socket layer, and is then copied to the application by a system call.

AF_XDP is a Linux socket type that lets a small eBPF program, running inside the network driver (XDP), hand selected packets directly to a user-space program, skipping most of that path [1], [2]. This project asks a practical question: *if the client-facing side of a real recursive resolver is moved from a UDP socket to AF_XDP, how much faster and cheaper does it become on ordinary hardware?*

We built a complete recursive resolver and gave it two interchangeable client front-ends: one that uses a UDP socket and one that uses AF_XDP. Both link exactly the same resolution engine, so any measured difference comes only from the packet path. Our contributions are:

- a recursive resolver that performs iterative resolution from the root servers, with caching, CNAME chains, negative answers and glueless delegations;
- an AF_XDP front-end with an XDP steering program, user-space frame construction and ring management (copy mode);
- an offline test environment with mock root, TLD and authoritative servers, and a 23-case functional test suite;
- measurements on a virtual (veth) link and on a physical Intel I219-LM NIC, including a step-by-step diagnosis of the bottleneck in the physical setup.

II. Problem Statement

Problem. Build a DNS resolver whose client queries are received and answered through AF_XDP instead of the kernel socket path, and measure how this changes throughput, latency and CPU cost compared with an otherwise identical socket-based resolver.

- **Input:** DNS queries over UDP/IPv4 for A or AAAA records, with the recursion-desired flag (RD) set.
- **Output:** a correct answer found by iterative resolution (addresses, a CNAME chain, NXDOMAIN, or an empty NOERROR for "no such record type"), or SERVFAIL after a timeout.
- **Assumptions:** Linux with AF_XDP support; clients on the same Ethernet segment as the resolver; one NIC receive queue.
- **Constraint:** only consumer hardware was available: laptop NICs whose drivers do not support zero-copy AF_XDP.
- **In scope:** A/AAAA resolution, caching, NOTIMP for other types, generic and native XDP, copy mode, IPv4.
- **Out of scope:** zero-copy mode, IPv6 transport, TCP and EDNS0, DNSSEC validation, multi-queue scaling.

Gap. AF_XDP is usually evaluated with simple programs that drop, echo or forward packets [3]. Such tests show the raw packet rate, but they keep no state between packets. A resolver is different: it must hold each client query while it talks to other servers, then build a correct reply later. This project measures AF_XDP inside such an application, on hardware without zero-copy support.

III. Project Objectives

O1. **Correctness.** Both resolvers pass all 23 functional checks (plain answers, CNAME chains within and across zones, NODATA, NXDOMAIN, glueless delegation, delegation loop, server timeout and cache hits) and silently drop all 7 malformed test packets.

O2. **AF_XDP data path.** Client queries are received on the AF_XDP RX ring and answered on the TX ring, with zero kernel-side AF_XDP drops.

O3. **Fair comparison.** Both binaries use the same resolution and wire-format code; only client I/O differs.

O4. **Measurement.** Report maximum throughput (5 runs), latency at a fixed load of 150k queries/s (3 runs) and CPU time per query, for three configurations: veth with generic XDP, veth with native XDP, and a physical NIC with generic XDP.

O5. **Reproducibility.** Every result can be regenerated from scripts in the repository.

IV. Architecture and Design

We present the design in the order it was built: a plain socket resolver first, then one piece at a time until the AF_XDP path is complete.

A. Step 1: a socket-based recursive resolver

A client sends a query with RD=1, meaning "find the full answer for me". The resolver then performs *iterative resolution*: it asks a root server, which refers it to the servers of the top-level domain (for example `com`), which refer it to the domain's authoritative servers, which return the answer [4], [5]. Each referral names the next servers and usually includes their addresses ("glue"). The baseline, `socket_resolver`,

receives client queries with `recvfrom()` and replies with `sendto()`. For every packet the kernel allocates a socket buffer (SKB), processes IP and UDP, finds the socket and copies the payload to user space.

B. Step 2: separating the engine from packet I/O

A fair comparison needs identical DNS work in both resolvers. We therefore split the program into three layers (Table I). `dns.c` reads and writes the DNS wire format; `resolver.c` holds the resolution logic and the cache; a small front-end moves packets. The front-end passes each query to the engine together with a *client context*, which is whatever it needs to address the reply later, and the engine calls the front-end back when the answer is ready, possibly many round trips later. For the socket front-end the context is the client's IP address and port; for the AF_XDP front-end it is the query's Ethernet, IPv4 and UDP headers.

TABLE I
CODE LAYERS (LINES OF CODE IN THE FINAL BUILD)

Layer	File	Role
Wire format	`dns.c` (331)	parse/build messages, name compression, query validation, `--local` zone
Engine	`resolver.c` (716)	iterative resolution, TTL cache, NXDOMAIN/NODATA, glueless NS, timeouts
Socket I/O	`socket_resolver.c` (101)	`recvfrom` / `sendto`, epoll loop
AF_XDP I/O	`afxdp_resolver.c` (559)	UMEM, four rings, frame construction
Steering	`xdp_kern.c` (75)	XDP program: UDP/53 to AF_XDP, rest to stack

C. Step 3: steering packets with XDP

XDP runs a small eBPF program in the network driver, before the kernel builds an SKB [1]. Our program, `xdp_kern.c`, checks the Ethernet, IPv4 and UDP headers. Packets with UDP destination port 53 are redirected through an `XSKMAP` to the AF_XDP socket; every other packet is passed to the normal stack (`XDP_PASS`). This keeps ARP, ping and SSH working, and it makes the steering rule an *externalised policy*: changing which traffic takes the fast path only requires changing this program, not the resolver.

D. Step 4: the AF_XDP socket

An AF_XDP socket shares a memory area, the *UMEM*, with the kernel. The UMEM is divided into fixed-size frames (4096 frames of 4 KiB). Four rings pass frame addresses between the program and the kernel [2]: the program puts empty frames on the **FILL** ring; the kernel writes received packets into them and posts them on the **RX** ring; the program posts finished replies on the **TX** ring; and the kernel returns sent frames on the **COMPLETION** ring. We give half of the frames to receiving and keep the other half as a pool for replies. Because packets arrive as raw Ethernet frames, the front-end builds every reply itself: it swaps the MAC addresses, IP addresses and UDP ports, sets the length fields and computes the IPv4 header checksum [8]. The UDP checksum is set to zero, which IPv4 permits [7].

E. Step 5: upstream traffic stays on the kernel stack

Queries to root, TLD and authoritative servers use an ordinary UDP socket in both binaries. Their replies arrive on an ephemeral port, not port 53, so the XDP program passes them to the kernel. This avoids re-implementing ARP and routing for Internet destinations, and it costs little: an upstream round trip takes milliseconds, while the client-path saving is measured in microseconds. The resolver still does all the iterative work itself; it never forwards to another resolver. Fig. 1 shows the complete design.

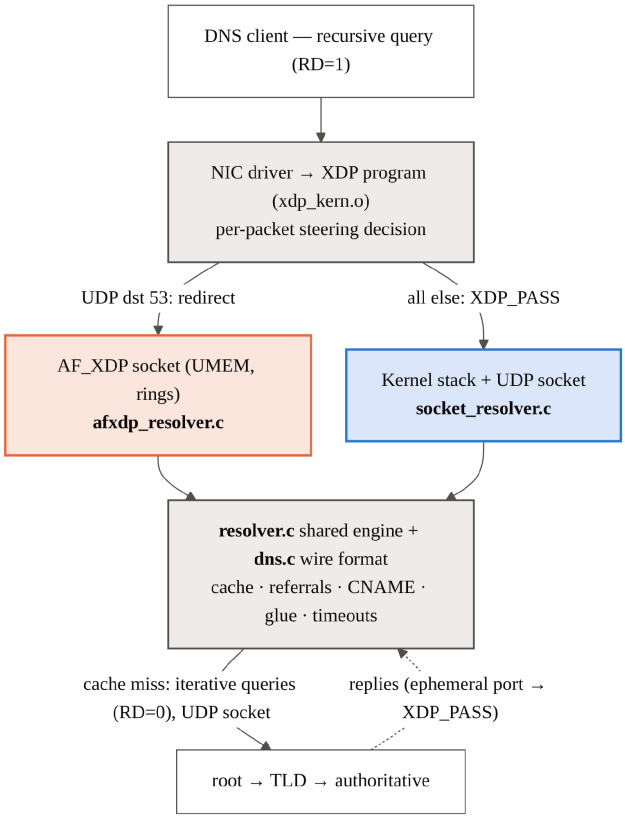


Fig. 1. Final architecture. The XDP program decides per packet which path a query takes; both front-ends feed the same engine, which resolves iteratively over an ordinary UDP socket and replies on the path the query arrived on.

F. Design choices and trade-offs

Why not zero-copy? In zero-copy mode the NIC writes packets directly into the UMEM, removing the last copy. This needs driver support that only some server NICs have (for example Intel `i40e`/`ice` or Mellanox `mlx5`) [2]. Our hardware could not provide it: the laptop's Intel I219-LM (`e1000e` driver) and its Wi-Fi (`iwlwifi`) do not even support native XDP, and `veth` supports native XDP but not zero-copy. We therefore use copy mode (`XDP_COPY`), which still bypasses the IP, UDP and socket layers but copies each packet once into the UMEM. All results are labelled as copy mode. Table II lists the other main choices.

TABLE II
MAIN DESIGN CHOICES

Choice	Alternative	Reason
Copy mode	Zero-copy	no driver support on available NICs
Measure generic and native XDP (`--skb`/`--drv`)	Let the kernel choose	the mode must be controlled and recorded; native is unavailable on `e1000e`
One shared engine	Two copies of the logic	identical DNS work by construction
Upstream on a UDP socket	Upstream on AF_XDP	ARP/routing for free; upstream latency dominates anyway
One RX queue	One socket per queue	simple; limits scaling to one core
`poll()` wake-ups	Busy polling	low idle CPU; costs one system call per wake-up
`--local` mode for benchmarks	Benchmark recursion	removes Internet delay so only the I/O path is measured

V. IMPLEMENTATION

The resolver is written in C (gcc, `-O3`); the XDP program in restricted C compiled with `clang -target bpf`; socket setup uses libbpf and the AF_XDP helpers of libxdp [9]. The mock DNS servers are Python 3 with dnspython, and the test tools are Bash scripts. Code: https://github.com/Ankit-matrix/AF-XDP_DNS-Resolver (commit `b0e9f6c`).

A. AF_XDP event loop

The loop below is the core of `afxdp_resolver.c`. It processes up to 64 frames per wake-up and sends replies in batches, which is where most of the per-packet saving comes from.

```
loop:
  poll(xsk_fd, upstream_fd, 100 ms)
  move COMPLETION-ring frames -> TX frame pool
  if upstream readable: resolver_handle_upstream()
  resolver_tick()               # retries, timeouts
  n = peek RX ring (max 64 frames)
  for each frame:
    check Ethernet/IPv4/UDP and dst port 53
    resolver_handle_query(dns, headers)
      -> xsk_reply(): free frame, copy headers,
         swap addresses, lengths, IP checksum
  flush replies to TX ring; kick kernel
  return RX frames to FILL ring
```

B. Iterative resolution in the engine

```
resolve(name, type):
  answer from cache (following cached CNAMEs)
  servers = 13 root servers, random start
  send query (RD=0, random TXID) to next server
  on reply (check port 53, server IP, TXID, question):
    NXDOMAIN       -> negative-cache, reply
    answer         -> cache, reply with CNAME chain
    CNAME only     -> name = target, restart at root
    SOA, no NS     -> NODATA: cache, empty NOERROR
    NS referral    -> servers = glue addresses;
                      no glue: child query for NS name
  1 s per server, then next; 5 s total -> SERVFAIL
```

Negative answers are cached for the SOA-derived time of RFC 2308 [6]. A referral is accepted only if it moves strictly down the tree toward the queried name, which stops delegation loops.

C. Problems found and fixed

1) **Random start-up failure.** libxdp does not initialise the ring's cached producer index, and the original test code reserved 4096 FILL entries on a 2048-entry ring. Start-up worked or failed depending on stack contents. Fix: zero-initialise the rings and reserve exactly the FILL size.
2) **Replies silently dropped at `-O3`.** The checksum routine read the IP header through a `uint16_t` pointer, breaking C's strict-aliasing rule; GCC moved a store past the read. tcpdump on the client showed:

```
IP (... length 82, bad cksum e9a7 (->e9a8)!)
  10.0.0.1.53 > 10.0.0.2.51602: [no cksum]
  ... A? github.com. 1/0/0 github.com. A 140.82.121.3
```

The fix reads the header byte by byte; all 12 combinations of optimisation level, mode and XDP type then produced valid checksums.

3) **CNAME and NODATA errors** were exposed by the mock hierarchy: the reply named the CNAME target in its question section, so `dig` rejected it, and "no such record type" was returned as SERVFAIL. Both were fixed once in the shared engine.
4) **Measurement pitfalls.** A leftover resolver process kept port 53 (both had `SO_REUSEADDR`), so CPU was recorded for an idle instance; and placing dnsperf on the resolver's hyper-thread sibling lowered the socket baseline from 310k to 219k queries/s. The benchmark script now refuses to start if two instances run, and pins dnsperf off the resolver's physical core.

VI. TESTBED SETUP

TABLE III
HARDWARE AND SOFTWARE

Item	Value
Resolver host	Dell Latitude 5420; 4 cores / 8 threads (CPU 2 and CPU 6 share core 2); <CPU model>, <RAM>
OS	Ubuntu 24.04 LTS, Linux 7.0.0-38-generic; `performance` governor
NICs	Intel I219-LM rev 20 (`e1000e`), 1 GbE; `veth` pair
Client host	Arch Linux, Linux 7.2.9-arch1-1, 16 logical CPUs, Realtek NIC (`r8169`)
Tools	dnsperf 2.14.0 [10], sysstat 12.6.1 (pidstat, mpstat), ethtool, tcpdump, dig
Build	gcc `-O3`, clang `-target bpf`, libbpf, libxdp

A. Network topologies

- **veth testbed** (`testbed.sh`): network namespace `dnstest` with `veth-client` (10.0.0.2) linked to `veth-dns` (10.0.0.1) on the host; one queue; checksum offload disabled so frames reach AF_XDP with real checksums.
- **Physical link**: laptop `enp0s31f6` (192.168.50.1) cabled directly to the client's `enp7s0` (192.168.50.2) at 1 Gb/s full duplex; dnsperf runs on the client over SSH.
- **Mock DNS hierarchy** (`mock_net.sh`): a namespace holding the 13 root-server addresses, a TLD server for `com`/`org` and three authoritative servers (`mock_dns.py`); host routes send the resolver's upstream queries there, so the resolver runs unmodified and offline.

B. Workloads and parameters

- `payload_local.txt`: 8 A queries matching the built-in zone; used for all benchmarks in `--local` mode.

- `payload_recursive_mock.txt`: 5000 queries, of which 4000 are unique uncached names (each a full root-TLD-authoritative walk) and 1000 are repeats of the test names.
- Benchmarks: resolver pinned to CPU 2, dnsperf on CPUs 0,1,3,4,5,7; `-c 8 -l 30`; throughput at `-Q 2000000` (5 runs), latency at `-Q 150000` (3 runs), 30 s pauses; runs with lost queries excluded. On the physical link dnsperf used `-T 4`.
- I219 tuning for the final runs: NIC rings raised from 256 to 4096 entries and the NIC interrupt moved to CPU 1 (Section VII-C).
- Resolver constants: UMEM 4096 x 4 KiB, AF_XDP rings 2048, batch 64, 1 s per-server timeout, 5 s query deadline, 8192 cache entries.

VII. Evaluation

A. Functional correctness

The AF_XDP receive path was first checked on the laptop with `xsk_rx_test`: a port-53 query appeared in user space with the expected addresses, while a query to port 5300 was passed to the kernel, which answered "connection refused".

```
$ sudo ./xsk_rx_test veth-dns
[1] ETH 02:b5:66:ba:bd:72 -> 56:ae:da:dc:ab:eb len=93
    IP  10.0.0.2 -> 10.0.0.1 proto=17 tot_len=79
    UDP 53196 -> 53 len=59
```

The resolver answered over both paths, and the attach mode was always checked before measuring (laptop output):

```
$ sudo ip netns exec dnstest dig @10.0.0.1 \
      github.com A +short
140.82.121.3
$ sudo ip netns exec dnstest dig @10.0.0.1 \
      nothere.com A | grep status
;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN
$ ip -d link show veth-dns | grep -o "xdp[a-z]*"
xdpgeneric          # --skb    ("xdp" with --drv)
```

Recursion was then tested against the mock hierarchy on the final build (development VM, Ubuntu 24.04, kernel 6.18; the scripts are the ones in the repository). All three variants, `socket_resolver`, `afxdp_resolver --skb` and `afxdp_resolver --drv`, passed all 23 checks. An excerpt:

```
$ sudo ./test_matrix.sh
PASS cold  www.github.com    A    -> NOERROR  4 ms
     www.github.com. 300 IN CNAME github.com.
     github.com. 300 IN A 140.82.121.3
PASS cold  github.com       AAAA -> NOERROR  4 ms
PASS cold  nope.google.com   A    -> NXDOMAIN 4 ms
PASS cold  www.glueless.com  A    -> NOERROR  4 ms
PASS cold  www.selfloop.com  A    -> SERVFAIL 4 ms
PASS cached www.google.com   A    -> NOERROR  0 ms
PASS timeout www.dead.com    A    -> SERVFAIL 1004 ms
== 23 passed, 0 failed
```

The mock servers' log confirms real iterative resolution. For the glueless case, the resolver first resolves the nameserver's own name, then asks that server:

```
193.0.14.129   www.glueless.com.  A   (root)
192.5.6.30     www.glueless.com.  A   (TLD: no glue)
193.0.14.129   ns1.elsewhere.org. A   (child query)
192.5.6.30     ns1.elsewhere.org. A
203.0.113.53   ns1.elsewhere.org. A
203.0.113.53   www.glueless.com.  A   (answer)
```

Under the 5000-query recursive load at 1000 queries/s every variant completed all queries with none lost (Table IV). On exit, `afxdp_resolver` reported `RX=10023 TX=10023` and all kernel AF_XDP drop counters at zero (`rx_ring_full=0`, `fill_empty=0`). The malformed-packet suite produced 7 silent drops and one answered control query for both resolvers. ASan/UBSan builds reported no errors.

Table IV
Recursive load: 5000 queries, mock hierarchy

Resolver	Completed / lost	Avg latency, cold cache	Avg latency, warm cache
socket	5000 / 0	1.76 ms	0.12 ms
AF_XDP, generic	5000 / 0	2.48 ms	0.11 ms
AF_XDP, native	5000 / 0	1.67 ms	0.19 ms

Response codes in every run: NOERROR 1000, NXDOMAIN 4000. Cold-cache latency is dominated by the upstream walk, not by the client path.

B. Performance on the veth testbed

Table V and Fig. 2 give the maximum throughput with one resolver core. AF_XDP raised throughput by 1.70x (generic XDP) and 1.58x (native XDP). The socket resolver spent about 90 % of its CPU time in the kernel (`sys`), which is the cost AF_XDP avoids:

```
$ pidstat -u -p $(pgrep -x socket_resolver) 1 20
Average:  %usr 8.90  %system 89.95  %CPU 98.85
```

Each run produced one summary line (`code/results/summaries/veth.txt`; some fields omitted here):

```
sock tput r1 qps=306015 lat_us=305 lost=0 cpu=99.08
sock tput r2 qps=312640 lat_us=303 lost=0 cpu=99.32
A    tput r2 qps=539620 lat_us=7   lost=0 cpu=99.52
sock lat  r1 qps=149999 lat_us=11  lost=0 cpu=69.84
A    lat  r2 qps=150000 lat_us=9   lost=0 cpu=59.76
```

Table V
veth testbed, --local mode

Configuration	Max QPS (mean ± s.d.)	CPU / query at max	Latency at 150k	CPU / query at 150k
socket	310.3k ± 2.6k	3.20 µs	11.0 µs	4.65 µs
AF_XDP generic	527.2k ± 13.9k	1.88 µs*	10.3 µs	3.99 µs
AF_XDP native	490.6k ± 3.4k	2.02 µs	9.0 µs	3.97 µs

* runs 2–5; run 1 recorded only 72.8 % CPU at normal throughput, which we attribute to the start of the sampling window.

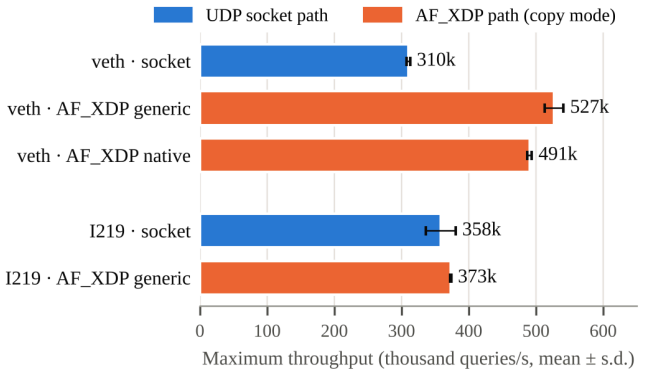


Fig. 2. Maximum throughput on one resolver core. Error bars show the standard deviation over runs (5 on veth, 4 on the I219 after excluding runs with lost queries).

Two results need explanation. First, generic XDP was slightly faster than native XDP on veth. Packets on a veth pair are already SKBs, created by the sender's stack, so native

mode must convert them to XDP buffers and saves nothing; we regard this as the most likely cause, but did not profile it. Second, at the fixed load of 150k queries/s the saving shrinks to 14–15 % per query (Fig. 3), and whole-machine CPU did not fall (2.36 cores for sockets versus 2.47–2.54 for AF_XDP). With fewer packets per wake-up, the fixed cost of each `poll()` and `sendto()` call is shared by fewer queries, and on veth part of the kernel work is charged to the sender's CPU.

C. Performance on the physical NIC

On the cabled I219-LM link both resolvers first stopped at about 377k queries/s with identical latency, while `afxdp_resolver` used only half a core, so the resolver was not the limit. Native XDP could not be attached:

```
$ sudo ./afxdp_resolver --local enp0s31f6 --drv
libbpf: Kernel error message: Underlying driver does
        not support XDP in native mode
bpf_xdp_attach: Operation not supported
```

We first suspected dnsperf's default limit of 100 outstanding queries: by Little's law [11], 100 / 258 µs gives about 388k queries/s. Raising the limit disproved this (Table VI): throughput stayed flat while latency grew in proportion, the signature of a fixed-rate stage with a queue in front of it.

TABLE VI
EFFECT OF DNSPERF OUTSTANDING-QUERY LIMIT (-Q)

-q	Throughput	Average latency
100	374.7k	0.26 ms
500	376.7k	1.32 ms
2000	369.5k	4.86 ms
5000	369.7k	4.85 ms

The laptop NIC's counters showed that its 256-entry receive ring was running out of buffers, and that its interrupt shared a physical core with the resolver:

```
$ ethtool -S enp0s31f6   # before/after a 10 s run
<   rx_no_buffer_count: 70960
>   rx_no_buffer_count: 100739
<   tx_restart_queue: 70
>   tx_restart_queue: 149
$ lscpu -e=CPU,CORE | awk 'NR==1||$1==2||$1==6'
CPU CORE
  2    2
  6    2
```

We raised the rings to the hardware maximum and moved the interrupt to CPU 1. A 10 s test then showed no further buffer shortages, but the same throughput:

```
$ ethtool -g enp0s31f6   # max RX/TX 4096, current 256
$ sudo ethtool -G enp0s31f6 rx 4096 tx 4096
$ echo 1 | sudo tee /proc/irq/172/smp_affinity_list
$ qt     # 10 s dnsperf + NIC counter diff
  Queries lost:         0 (0.00%)
  Queries per second:   376359.300663
  Average Latency (s):  0.000258
  (rx_no_buffer_count, tx_restart_queue unchanged)
```

The limit had moved to the client. Its transmit queue-discipline *requeue* counter rose in every run (by 21k–115k), which happens when the NIC's transmit ring is full, while its receive side reported no misses:

```
client$ tc -s qdisc show dev enp7s0
 Sent 18743424063 bytes 258524654 pkt
   (dropped 0, overlimits 0 requeues 519922)
client$ ethtool -S enp7s0 | grep -iE 'miss|drop|err|fifo'
     tx_errors: 2
     rx_errors: 3
     rx_missed: 0
     align_errors: 0
     tx_underrun: 0
```

Both resolvers therefore reached the same client-side ceiling, and on this link the useful comparison is CPU cost. The final runs (`code/results/summaries/i219_final.txt`, fields omitted) show the difference directly:

```
sockC tput r2 qps=376797 cpu=82.24 hotcore=CPU2 83%
C     tput r2 qps=374235 cpu=37.88 hotcore=CPU1 71%
sockC lat  r1 qps=150000 lat_us=119 cpu=34.56
C     lat  r1 qps=150000 lat_us=117 cpu=15.28
```

TABLE VII
INTEL I219-LM, GENERIC XDP, --LOCAL MODE

Resolver	Max QPS (mean ± s.d.)	Resolver CPU at max	CPU / query at 150k	Whole machine at 150k
socket	358.4k ± 22.1k	84.8 %	2.36 µs	1.28 cores
AF_XDP	373.2k ± 0.8k	37.4 %	1.03 µs	1.04 cores

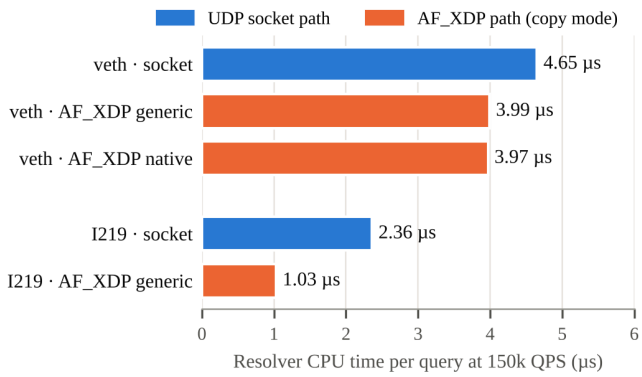


Fig. 3. Resolver CPU time per query at an equal load of 150k queries/s (mean of 3 runs).

On real hardware AF_XDP cut the resolver's CPU time per query by 56 % (2.36 to 1.03 µs) and whole-machine CPU by 19 % at the same load. At maximum load the socket resolver's interrupt core reached 88–90 % soft-interrupt time and its throughput varied by ±22k between runs, whereas AF_XDP served the same rate at 37 % of a core with a spread of ±0.8k. Latency at 150k queries/s was the same for both (about 120 µs), because on this path it is dominated by the wire, NIC interrupt moderation and the client's queues.

D. Discussion

The saving comes from skipping SKB handling, the IP and UDP layers and the socket layer, and from processing up to 64 packets per wake-up instead of one system-call pair per packet. Copy mode does not remove the kernel entirely: at saturation `afxdp_resolver` still spent about 82 % of its time in `sys`, because in copy mode transmission runs inside the `sendto()` wake-up and every `poll()` is a system call.

Zero-copy and busy polling are therefore expected to reduce the cost further, but we could not measure them on this hardware.

VIII. Summary

We built a recursive DNS resolver with two client paths, a standard UDP socket and AF_XDP, sharing one iterative resolution engine, and verified it with a mock DNS hierarchy and a 23-case test suite. On a veth link the AF_XDP path delivered 1.58–1.70x the throughput of the socket path on one core. On a physical Intel I219-LM link, where a single-queue client NIC capped both resolvers at about 375k queries/s, AF_XDP served the same load with less than half the CPU per query (1.03 versus 2.36 µs). The project also showed that the measurement setup matters as much as the code: NIC ring sizes, interrupt placement, hyper-thread siblings and leftover processes each changed the results.

Limitations: copy mode only; one receive queue and one resolver core; IPv4 only; replies assume the client is on the same Ethernet segment; no EDNS0, TCP fallback or DNSSEC; a single upstream socket with a fixed source port, so spoofing protection rests on the 16-bit transaction ID.

Future work: measure zero-copy on a supported NIC (e.g. `ice`) with two hosts; use one AF_XDP socket per receive queue with RSS for multi-core scaling; add busy polling; randomise the upstream source port; add EDNS0 and TCP fallback.

IX. Reproducibility

Repository: https://github.com/Ankit-matrix/AF-XDP_DNS-Resolver (commit `b0e9f6c`). Code is in `code/src/`, test scripts and query lists in `code/local_offline_testing/`, and every raw output and summary used in this report in `code/results/`. Commands assume Ubuntu 24.04.

1. Build (run resolvers from `code/src/`, where `xdp_kern.o` is loaded):

```
$ git clone https://github.com/Ankit-matrix/\
      AF-XDP_DNS-Resolver
$ cd AF-XDP_DNS-Resolver/code/src
$ make   # socket_resolver afxdp_resolver
         # xdp_kern.o xsk_rx_test
```

2. Functional tests (Section VII-A):

```
$ cd ../local_offline_testing
$ sudo ./testbed.sh setup && sudo ./mock_net.sh up
# terminal 2, in code/src:
$ sudo ./afxdp_resolver veth-dns --skb
# terminal 1:
$ sudo ./test_matrix.sh      # 23 passed, 0 failed
$ sudo ip netns exec dnstest dnsperf -s 10.0.0.1 \
    -d payload_recursive_mock.txt -n 1 -Q 1000 -c 8
$ sudo ./mock_net.sh down
```

3. veth benchmark (Table V; repeat with `./socket_resolver --local` and with `--drv`):

```
$ sudo taskset -c 2 \
    ./afxdp_resolver veth-dns --local --skb
$ sudo ip netns exec dnstest taskset -c 0,1,3,4,5,7 \
    dnsperf -s 10.0.0.1 \
    -d ../local_offline_testing/payload_local.txt \
    -l 30 -c 8 -Q 2000000        # or -Q 150000
$ pidstat -u -p $(pgrep -x afxdp_resolver) 5 5
```

4. Physical NIC (Table VII): configure 192.168.50.1/24 on the resolver and 192.168.50.2/24 on the client, then:

```
$ sudo ethtool -G enp0s31f6 rx 4096 tx 4096
$ for i in $(grep enp0s31f6 /proc/interrupts \
      | cut -d: -f1); do echo 1 | sudo tee \
      /proc/irq/$i/smp_affinity_list; done
$ sudo taskset -c 2 \
    ./afxdp_resolver enp0s31f6 --local --skb
client$ dnsperf -s 192.168.50.1 -d payload_local.txt \
          -l 30 -T 4 -c 8 -Q 2000000
```

5. Raw data. Each run's dnsperf, pidstat and mpstat output is stored as

```
code/results/<testbed>/<resolver>/
    {throughput,latency_150k}/runN_*.txt
```

with one line per run in `code/results/summaries/` and the native-XDP failure and per-core CPU samples in `code/results/diagnostics/`.

References

[1] T. Høiland-Jørgensen, J. D. Brouer, D. Borkmann, J. Fastabend, T. Herbert, D. Ahern, and D. Miller, "The eXpress data path: Fast programmable packet processing in the operating system kernel," in Proc. 14th Int. Conf. Emerging Networking EXperiments and Technologies (CoNEXT '18), Heraklion, Greece, 2018, pp. 54–66, doi: 10.1145/3281411.3281443.

[2] The Linux Kernel Documentation, "AF_XDP," Documentation/networking/af_xdp.rst. [Online]. Available: https://docs.kernel.org/networking/af_xdp.html

[3] M. Karlsson and B. Töpel, "The path to DPDK speeds for AF_XDP," presented at Linux Plumbers Conf., Vancouver, Canada, Nov. 2018. [Online]. Available: https://lpc.events/event/2/contributions/99/

[4] P. Mockapetris, "Domain names – concepts and facilities," RFC 1034, Nov. 1987.

[5] P. Mockapetris, "Domain names – implementation and specification," RFC 1035, Nov. 1987.

[6] M. Andrews, "Negative caching of DNS queries (DNS NCACHE)," RFC 2308, Mar. 1998.

[7] J. Postel, "User Datagram Protocol," RFC 768, Aug. 1980.

[8] R. Braden, D. Borman, and C. Partridge, "Computing the Internet checksum," RFC 1071, Sep. 1988.

[9] XDP Project, "xdp-tools: libxdp and utilities for XDP." [Online]. Available: https://github.com/xdp-project/xdp-tools

[10] DNS-OARC, "dnsperf: DNS performance testing tool." [Online]. Available: https://github.com/DNS-OARC/dnsperf

[11] J. D. C. Little, "A proof for the queuing formula: L = λW," Operations Research, vol. 9, no. 3, pp. 383–387, 1961.

Appendix A: Team Work Distribution and Member Rating

A.1 Work Distribution

- Ratings are agreed by the whole team and are unique (no ties); scale 1–10 (10 = highest contribution).

#	Member Name	Roll No.	Tasks Done	Rating (1–10, unique)
1	Divisht	24110112	Testing and benchmarking (confirmed). He built the veth test setup and ran the functional tests and malformed-packet checks. He ran every benchmark on his Dell Latitude 5420 (veth and the Intel I219-LM): throughput, latency at 150k QPS, and CPU per query. He tuned the NIC, confirmed native XDP is unsupported on e1000e, and wrote up the results.	9
2	Ankit Saha	24110042	Code generation with Claude (confirmed). He planned the workflow and generated the code: the shared DNS module, the iterative resolver engine, the socket baseline, the XDP kernel program, the AF_XDP RX test, the AF_XDP resolver and the Makefile. He integrated everything in the repo, ran the architecture study with ChatGPT, and made the slide deck and README.	10
3	Jahan Zaib	24110140	Proposed: the original UDP forwarder baseline (epoll loop, transaction-ID state table, hardening fixes) and its malformed and spoofed packet tests.	8
4	Mateen Sharief	24110197	Proposed: AF_XDP concept study (UMEM, the four rings, XSKMAP, copy vs zero-copy), and review and verification of the generated code against it.	7
5	Adil Khan N	23110013	Partly confirmed: he co-planned the workflow with Ankit and owns the GitHub repo. Proposed: baseline UDP resolver co-development and repo setup.	6
6	Aditya Kumar	22110140	Proposed: documentation support, slide content review, and the client machine for the two-host NIC test.	5

Appendix B: AI Usage Details

B.1 Prompts

The most significant uses of AI assistance are listed below. Prompts and responses are shared as .md files in the `AI Prompts` folder of the repository.

#	AI Tool	Purpose
1	Claude (Anthropic)	Code review of the initial repository; finding the FILL-ring and strict-aliasing checksum bugs. Drafting and editing this report from the team's measured results
2	Claude (Anthropic)	Refactoring into the shared engine (`resolver.c`) with recursive mode in both front-ends; mock DNS hierarchy and test scripts
3	Claude (Anthropic)	Benchmark methodology and bottleneck diagnosis on the physical NIC (ring sizes, interrupt placement, client requeues),
5	ChatGPT (OpenAI)	Explaining AF_XDP architecture and packet-flow design; analysing UMEM frame management, the four-ring lifecycle, XDP redirection through XSKMAP, and NIC RX queue interactions. Comparing hybrid UDP/AF_XDP and fully AF_XDP recursive DNS architectures, including upstream query handling and implementation trade-offs.